

ContractGuard

Functional Specification

Justin Siak	22449184
Chunyang Wang	21115214

Project Supervisor: Graham Healy

25 Nov 2025

Table of contents

ContractGuard	1
Functional Specification	1
1. Introduction	3
1.1 Overview	3
1.2 Business Context	4
1.3 Glossary	5
2. General Description	6
2.1 Product / System Functions	6
1. API Specification Ingestion	6
2. Rule-Based Test Case Generation	6
3. LLM-Assisted Enhancements	7
4. Automated Test Execution	7
5. Dashboard and Result Visualisation	7
6. CI/CD Integration	7
2.2 User Characteristics and Objectives	8
User Objectives	8
Feasible Solutions for This Project Scope	8
2.3 Operational Scenarios	9
Uploading an API Specification	9
Generating and Running Tests Automatically	9
Handling Failed Tests	9
Using the LLM Chat Window	9
Reviewing Test History	9
Running Tests Automatically Through GitHub Actions	9
Submitting an Invalid or Incomplete Specification	9
2.4 Constraints	10
Technical and Performance Constraints	10
Resource and Platform Constraints	10
Integration Constraints	11
3. Functional Requirements	11
3.1 API Specification Upload and Parsing	11
3.2 Rule-Based Test Generation	12
3.3 LLM-Assisted Test Enhancement and Explanations	12
3.4 Automated Test Execution	13
3.5 Dashboard and Result Visualisation	13

3.6 Test History Storage and Retrieval	14
3.7 GitHub Actions Integration for CI/CD	14
3.8 LLM Chat Window	15
4. System Architecture	15
4.1 Client-Side	16
4.2 Web Frontend	17
4.3 Backend Service (FastAPI + Python)	17
4.4 Third-Party LLM Provider	18
4.5 Database	19
4.6 Containerisation	19
5. High-Level Design	19
5.1 Data Flow Diagram	19
DFD Level 0 (Context Diagram)	19
DFD Level 1A (Parsing & Test Generation)	20
DFD Level 1B (Test Execution & LLM Enhancement)	21
DFD Level 1C (CI/CD Workflow)	22
5.2 Sequence Diagram (Core Workflow)	22
5.3 Object Model Diagram	23
6. Preliminary Schedule	23
6.1 Preliminary Schedule GANTT chart	24
Short Resource Requirements Description	24
7. Appendices	24

1. Introduction

1.1 Overview

ContractGuard is a platform designed to automatically verify API contracts and assist developers in maintaining reliable API behaviour. Systems rely heavily on APIs, but ensuring that each endpoint behaves according to its OpenAPI/Swagger specification can be tiring as manual testing becomes slow and repetitive.

By using ContractGuard, users can upload API specifications through a web dashboard, and the system will parse the file, generate rule based tests, and execute them automatically. For complex schemas or unclear constraints,

ContractGuard integrates an LLM that assists in generating additional cases and producing human-readable explanations when failures occur. The platform also includes an LLM chat window, giving users a flexible space to ask questions, request explanations, or explore possible improvements. As LLM capabilities grow, we expect more functionalities such as advanced debugging assistance or automated schema refinement to become feasible within the platform.

ContractGuard operates as a standalone web application but also integrates with external tools. GitHub Actions support allows the platform to run tests automatically whenever new specifications are pushed in Github, making it suitable for lightweight CI/CD workflows.

Key Functions

- Automated API specification parsing
- Rule based test generation
- LLM assisted test enhancement
- Automated test execution
- Interactive dashboard
- LLM explanation and chat window
- CI/CD integration

1.2 Business Context

ContractGuard is developed as a student project but aims to solve a real gap in current development workflows. Existing automated API testing platforms require paid subscriptions or corporate accounts, and almost none of them combine automated contract verification with LLM. As a result, developers would need to manually search through past test results, compare behaviour, and interpret failures on their own.

ContractGuard provides a free and accessible alternative. It can support students, academic staff, small project teams, or early stage companies by offering automated API verification along with clear, LLM-generated insights.

1.3 Glossary

Term	Definition
API Contract	A formal description of an API's expected behaviour, structure, and constraints, typically defined using OpenAPI/Swagger.
OpenAPI / Swagger	A machine-readable specification format (JSON or YAML) used to describe REST APIs, including endpoints, parameters, schemas, and validation rules.
Schema	A structured definition describing the shape, types, and constraints of request/response bodies in an API specification.
Rule-Based Test Generation	Automatic creation of test cases based strictly on predefined rules derived from the API specification (e.g required fields, types, allowed values).
LLM (Large Language Model)	A third-party AI model (e.g OpenAI, Claude, local models) used to generate additional test cases, produce human-readable explanations, and support chat functionality.
LLM Adapter	A backend module that formats prompts, sends requests to the LLM provider, and processes model outputs for test enhancement or explanations.
CI/CD (Continuous Integration / Continuous Deployment)	Automated workflows (via GitHub Actions) that trigger tests whenever code is updated in a repository.

Webhook	A callback mechanism where GitHub Actions notifies ContractGuard that a new version of the API specification is available.
DAL (Data Access Layer)	A backend module responsible for structured storage and retrieval of specifications, test cases, execution history, and LLM explanations in PostgreSQL.

2. General Description

2.1 Product / System Functions

ContractGuard provides the core features that help users automatically verify API behaviour and give users a complete workflow for understanding failures, and keeping track of changes over time without needing expensive tools or complex setup.

Its main functions include:

1. API Specification Ingestion

- Users can upload OpenAPI [1] or Swagger files through the dashboard.
- The backend parses endpoints, parameters, request/response bodies, and schema constraints.
- Any parsing errors are reported clearly to identify invalid or incomplete specifications.

2. Rule-Based Test Case Generation

- Generates tests directly from the specification without requiring users to write scripts.
- Produces positive, negative and edge test cases based on types, required fields, allowed values, and schema rules. [2]

3. LLM-Assisted Enhancements

- For more complex or ambiguous schemas, the LLM will be called to suggest additional test cases.
- When tests fail, the LLM can provide human readable explanations to help users quickly understand the issue.
- A built-in chat window allows users to ask questions about the API behaviour, test results, or debugging steps.

4. Automated Test Execution

- The backend runs the generated tests using a structured test runner.
- Each execution includes logs, request/response details, and pass/fail outcomes.
- Results are stored for future reference.

5. Dashboard and Result Visualisation

- The frontend displays test history, success rates, failure causes, and LLM explanations.
- Users can navigate between past runs without searching manually through files or logs.

6. CI/CD Integration

- ContractGuard can integrate with GitHub Actions. [5]
- When API specifications are updated in a repository, tests run automatically, providing continuous validation.

These functions together give the users a complete workflow for verifying APIs, understanding failures, and keeping track of changes over time without needing paid tools or complex setup.

2.2 User Characteristics and Objectives

ContractGuard is intended for users who work with APIs but may not have access to expensive or restricted automated testing tools. These users include students learning backend development, academic staff building teaching or research systems, small project teams, and independent developers maintaining services. Their technical background usually covers basic API knowledge such as HTTP requests [4], JSON structures, and reading OpenAPI files but they may not be familiar with writing automated tests or setting up full CI/CD pipelines.

User Objectives

From the user's perspective, ContractGuard should enable them to:

- Verify API behaviour quickly without writing test scripts manually.
- Understand why an API fails, and have a quick look through test cases and test results, understanding failures through clear explanations instead of reading long logs.
- Keep track of test history without searching through local files or raw output.
- Use a free and accessible tool instead of relying on paid, company restricted services.
- Automate testing in CI/CD with minimal configuration.
- More advanced LLM debugging assistance such as suggesting potential fixes for failing endpoints.

Feasible Solutions for This Project Scope

Within the scope of this student project, ContractGuard focuses on:

- Upload based specification parsing
- Rule based and LLM assisted test generation
- Automated test execution
- Dashboard visualisation
- LLM chat window and fails explanations
- Basic GitHub Actions CI/CD integration

2.3 Operational Scenarios

Uploading an API Specification

The user uploads an OpenAPI or Swagger document through the dashboard. The system parses the file and displays a structured summary of detected endpoints, parameters, and schemas.

Generating and Running Tests Automatically

After parsing, the user starts a test run. The system generates rule-based test cases based on schema definitions and executes them. All requests, responses, and outcomes are recorded.

Handling Failed Tests

If a test fails, the system stores the failed request, the actual response, and the reason why it does not match the specification. The LLM may generate additional test cases to further demonstrate the issue and provide a clear explanation of what went wrong.

Using the LLM Chat Window

The user can open the LLM chat window to interact with the system. They may ask questions, explore ideas, or use it for additional assistance. The exact usage is flexible and not limited to debugging.

Reviewing Test History

The user opens the history page to view past test runs. The system displays previous results, timestamps, and associated explanations, allowing the user to compare different versions of their API over time.

Running Tests Automatically Through GitHub Actions

The user updates an API specification in a GitHub repository. GitHub Actions triggers ContractGuard to run the tests, and the updated results appear in the dashboard once the CI workflow completes.

Submitting an Invalid or Incomplete Specification

If the uploaded specification is missing required fields or contains invalid structure, the system reports parsing errors and stops the test generation process. Since many users may only be API consumers and cannot modify the original specification, the system suggests uploading a different version of the specification or using another API if available.

2.4 Constraints

The development of ContractGuard is subject to several practical constraints. As a student project, the system must remain lightweight, maintainable, and feasible within the available time and resources. The following constraints influence the design and implementation choices:

Technical and Performance Constraints

- **Execution Time:** Test generation and execution must remain reasonably fast. Since the platform may be used by students or small teams, long running jobs or heavy computations are avoided.
- **LLM Latency:** LLM generated explanations or additional test cases depend on external model response time. Slow responses can impact user experience.
- **Specification Complexity:** Very large or deeply nested API specifications may increase parsing time or exceed what the rule-based engine can handle efficiently.

Resource and Platform Constraints

- **Hosting and Cost:** The system is free to use, so all components will run on affordable or low-cost infrastructure. This limits the use of high cost LLM models or heavy compute workloads.
- **Hardware Limitations:** No special hardware is assumed. The system will operate on standard cloud or local machines without requiring GPUs or high memory servers.
- **Dashboard:** The web interface must work reliably in common browsers without requiring additional plugins or extensions.

Integration Constraints

- **API Accessibility:** ContractGuard relies on the tested API being reachable from the environment where the tests are executed. Private or inaccessible endpoints cannot be evaluated.
- **User Cannot Modify Specifications:** Many users may be API consumers rather than API authors, meaning that invalid or incomplete specifications cannot be fixed by them. The system can only report parsing errors and cannot assume that the users will correct the spec.
- **CI/CD Limitations:** GitHub Actions integration must use lightweight workflows. Resource heavy pipelines are not feasible for typical free usage.

3. Functional Requirements

The following functional requirements describe what ContractGuard must accomplish to support automated API contract verification. They are listed in order of importance and time order.

3.1 API Specification Upload and Parsing

Description

The system must allow users to upload an OpenAPI or Swagger specification and parse its content to extract endpoints, parameters, schemas, and constraints.

Criticality

High, all other functions depend on understanding the specification.

Technical Issues

- Handling invalid or incomplete specifications.
- Supporting differences between Swagger 2.0 and OpenAPI 3.
- Ensuring the parser can process nested or complex schemas.

Dependencies

Required before 3.2 (test generation), 3.3 (LLM explanation), and 3.4 (test execution).

3.2 Rule-Based Test Generation

Description

The system must generate structured test cases directly from the parsed specification using predefined rules (e.g required fields, type validation, allowed values).

Criticality

High, core functionality for automated verification.

Technical Issues

- Designing a consistent rule set that handles common schema patterns.
- Managing edge cases such as optional fields or polymorphic schemas.

Dependencies

Depends on 3.1 (API specification parsing). Generated test cases used for execution (3.4).

3.3 LLM-Assisted Test Enhancement and Explanations

Description

The system must be able to call an LLM to generate additional test cases for complex or unclear endpoints. When an initial test fails, the LLM should also produce extra test cases to re-run against the API, helping confirm the failure and illustrate the issue more clearly. The LLM must also provide readable explanations summarising why the behaviour does not match the specification.

Criticality

Medium-High, this significantly improves clarity and verification quality, though the core system can still operate without it.

Technical Issues

- LLM latency and rate limits.
- Ensuring prompts are structured enough to guide meaningful test generation.
- Managing conflicts when LLM generated cases differ from rule-based expectations.
- Avoiding excessive or redundant test regeneration that could slow down execution.

Dependencies

Depends on 3.2 (test generation) and 3.4 (test execution) to provide context for LLM driven enhancements.

3.4 Automated Test Execution

Description

The system must execute all generated test cases against the target API and record requests, responses, and pass/fail outcomes.

Criticality

High, essential to validate API behaviour.

Technical Issues

- Ensuring consistent execution speed.
- Handling APIs that return inconsistent or unexpected formats.
- Managing unreachable or unstable endpoints.

Dependencies

Requires 3.2 (test generation). Supports 3.3 (explanation), 3.5 (visualisation), and 3.6 (test history).

3.5 Dashboard and Result Visualisation

Description

The system must display test results in a clear and accessible interface, including pass/fail states, response data, and LLM explanations.

Criticality

Medium, improves usability but does not affect core test execution logic.

Technical Issues

- Managing large result sets.
- Presenting data in a readable format across different browsers.

Dependencies

Depends on 3.4 for test outcomes and 3.3 for LLM explanations.

3.6 Test History Storage and Retrieval

Description

The system must store previous test runs and allow users to revisit results at any time.

Criticality

High, valuable for tracking changes over time.

Technical Issues

- Database schema design for storing large test logs.
- Ensuring efficient retrieval without long load times.

Dependencies

Depends on 3.4 to generate data; supports 3.5 for visualisation.

3.7 GitHub Actions Integration for CI/CD

Description

The system must provide a lightweight GitHub Actions workflow that automatically triggers tests when a repository's API specification changes.

Criticality

Medium, important for continuous verification but not required for basic usage.

Technical Issues

- Ensuring workflow is simple enough for non-experts.
- Avoiding long-running or costly pipelines.

Dependencies

Depends on 3.1-3.4 for parsing, generation, and execution.

3.8 LLM Chat Window

Description

The system must allow users to interact with an LLM through a chat interface for general assistance, exploration, or additional context related to API behaviour.

Criticality

Low-Medium, useful for user experience but not a core testing requirement.

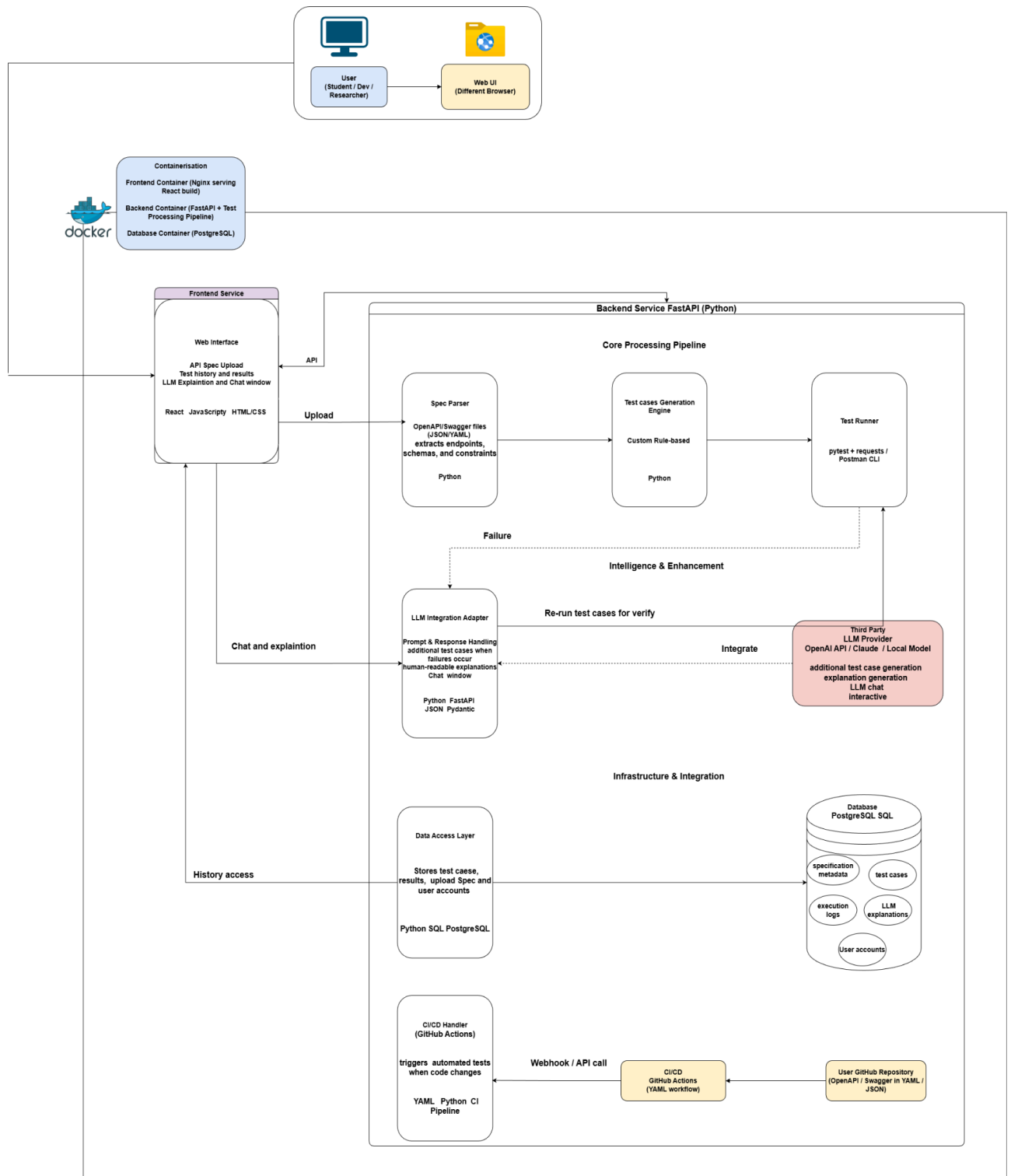
Technical Issues

- Managing LLM cost and usage limits.
- Ensuring a simple and stable chat interface.

Dependencies

Independent of 3.2 test generation but enhances 3.3.

4. System Architecture



4.1 Client-Side

Users interact with the system through their web browser, regardless of the device they are using. The application supports laptops, desktops, and mobile devices, allowing students, developers, and researchers to upload API

specifications, view test results, and access the LLM explanation features without installing additional software.

4.2 Web Frontend

The web frontend provides a clean and intuitive interface built with React. It allows users to:

- Upload OpenAPI/Swagger specification files
- Trigger automated test runs
- View test history and detailed results
- Read LLM-generated explanations
- Optionally use the integrated chat window for further queries

The frontend communicates with the backend via REST API calls and is served as static assets from an Nginx-based container.

4.3 Backend Service (FastAPI + Python)

The backend performs all core logic and coordinates processing across the system. It is organized into three main components:

Core Processing Pipeline

- **Spec Parser**
Parses OpenAPI/Swagger files (JSON/YAML), extracts endpoints, schemas, parameters, and validation rules.
- **Test Case Generation Engine**
Produces rule-based test cases from the specification, ensuring coverage of required fields, constraints, and edge cases.
- **Test Runner**
Executes generated test cases using pytest [5], Postman CLI, or HTTP requests [4]. It communicates with the target API and records

responses.

Intelligence & Enhancement

- **LLM Integration Adapter**

When failures occur, the backend forwards the failing cases and context to an LLM. The LLM generates additional test cases for verification and produces human-readable explanations to help users understand what went wrong.

- Provide chat window for many possibilities

Infrastructure & Integration

- **Data Access Layer (DAL)**

Stores specification metadata, generated test cases, execution logs, historical results, LLM explanations, and user data in PostgreSQL.

- **CI/CD Handler**

Receives webhook triggers from GitHub Actions when a specification changes in a linked repository, initiating a new automated test run.

4.4 Third-Party LLM Provider

The system integrates with third-party LLM services such as OpenAI [6], Claude [7], or local deployed models.

These models assist the backend in:

- Generating additional test cases when initial tests fail
- Producing readable, context-aware explanations
- Supporting optional interactive chat within the frontend

The integration is optional and modular, allowing different model providers to be plugged in without modifying the core pipeline.

4.5 Database

A PostgreSQL database [8] stores all persistent and historical information generated by the backend. This includes:

- Uploaded API specifications
- Generated test cases
- Test execution logs and results
- LLM-generated explanations
- User accounts and optional metadata

The database runs as a separate container within the Docker Compose environment.

4.6 Containerisation

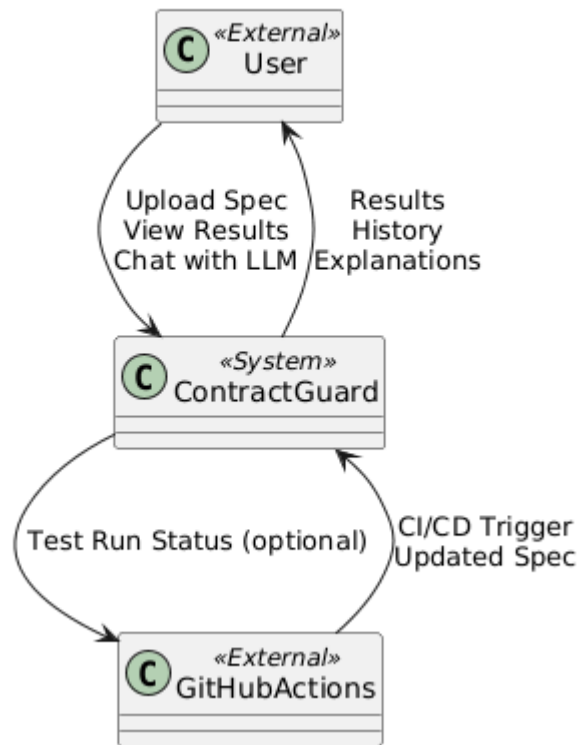
The entire system is deployed using Docker Compose, ensuring consistent environments and easy setup. The following containers are orchestrated:

- **Frontend container**
Nginx serves the built React application.
- **Backend container**
FastAPI application plus all processing modules (parser, test generator, test runner, LLM adapter, CI/CD handler).
- **Database container**
PostgreSQL database for storage and retrieval.

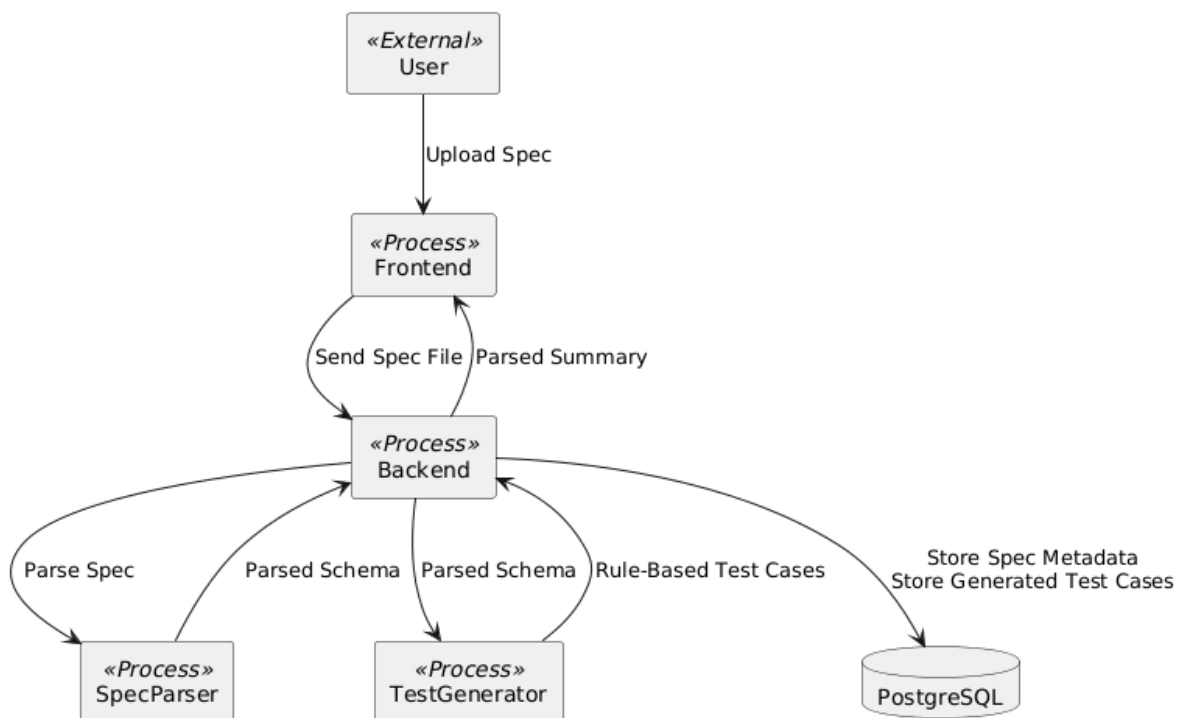
5. High-Level Design

5.1 Data Flow Diagram

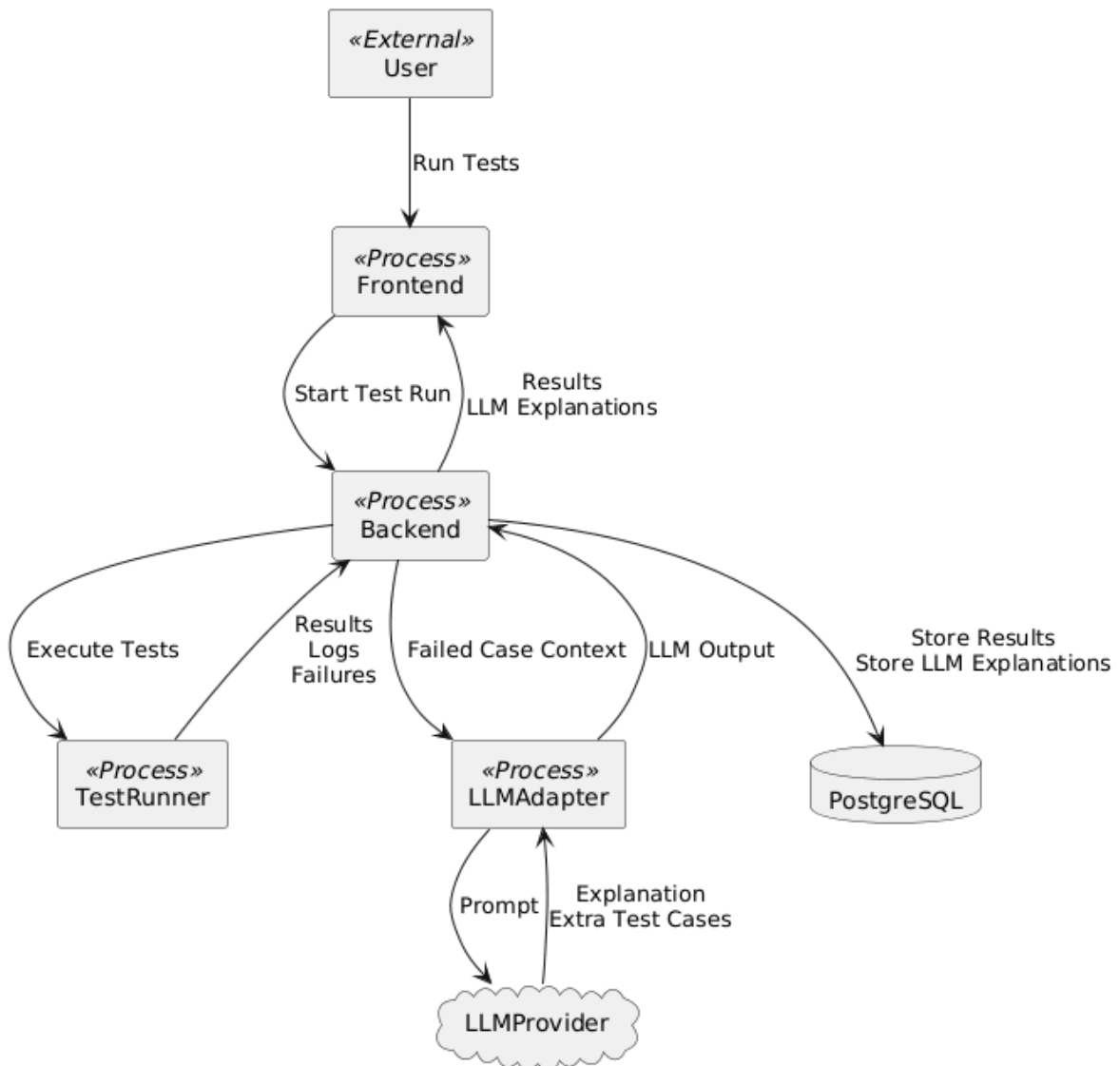
DFD Level 0 (Context Diagram)



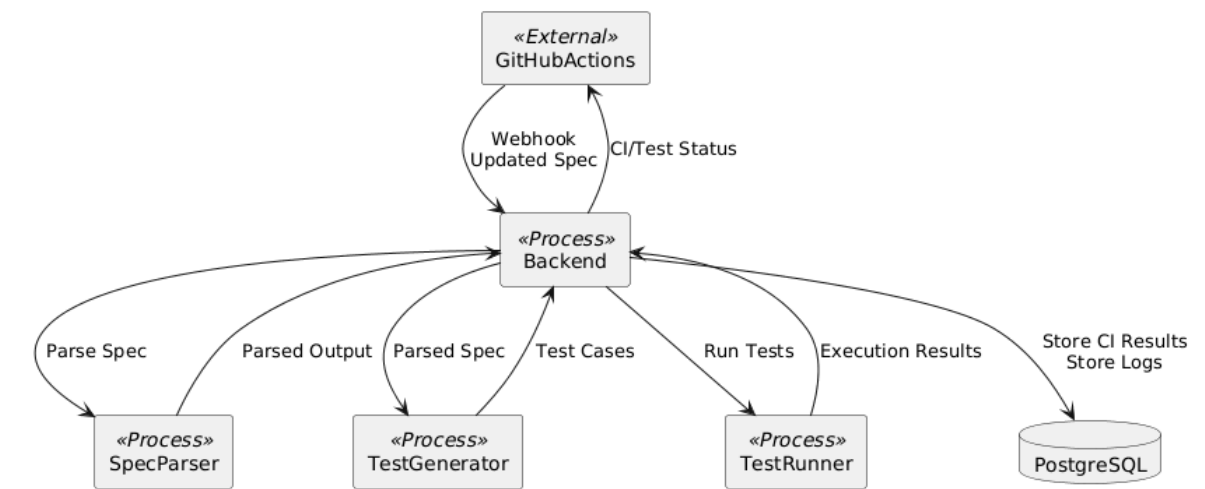
DFD Level 1A (Parsing & Test Generation)



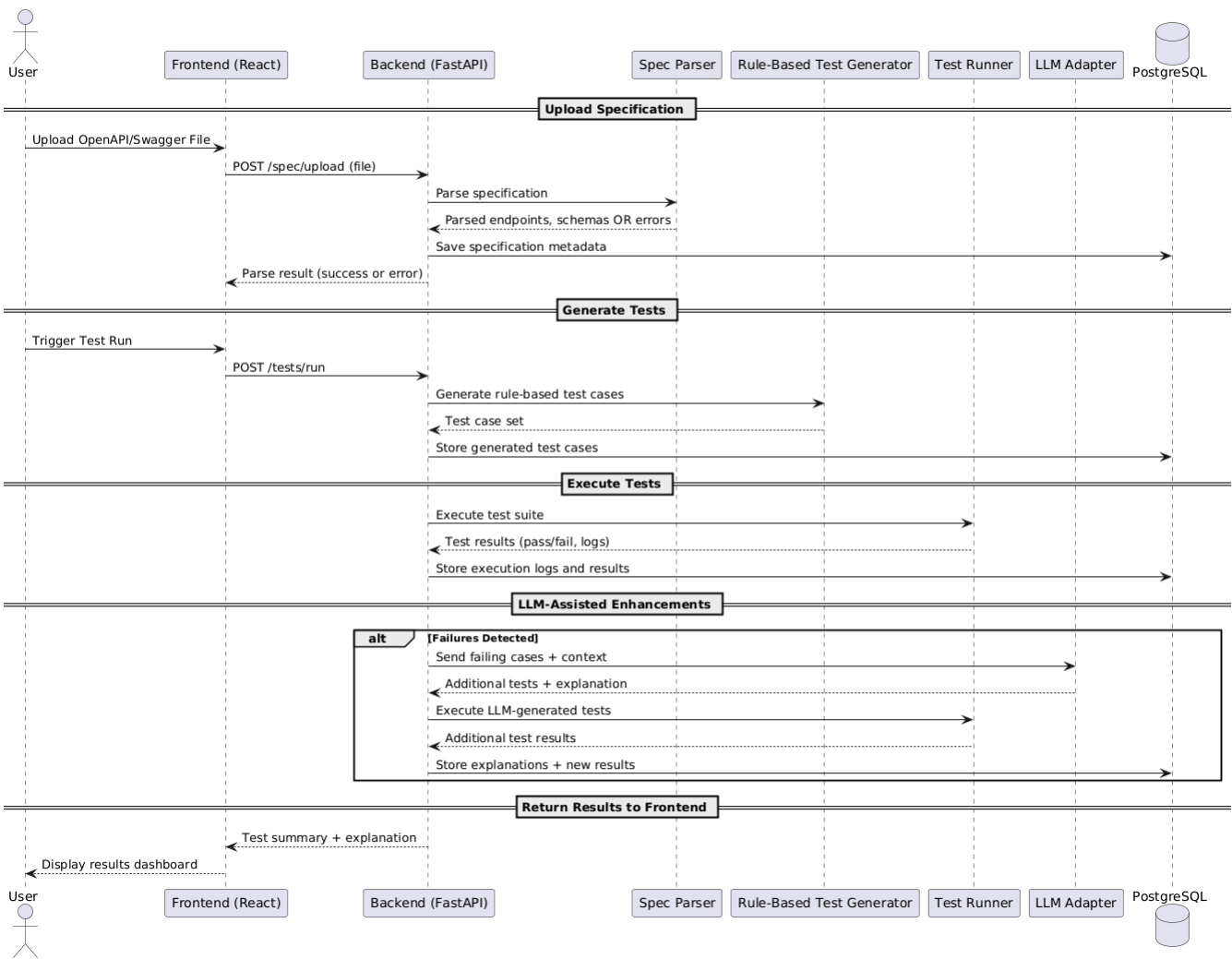
DFD Level 1B (Test Execution & LLM Enhancement)



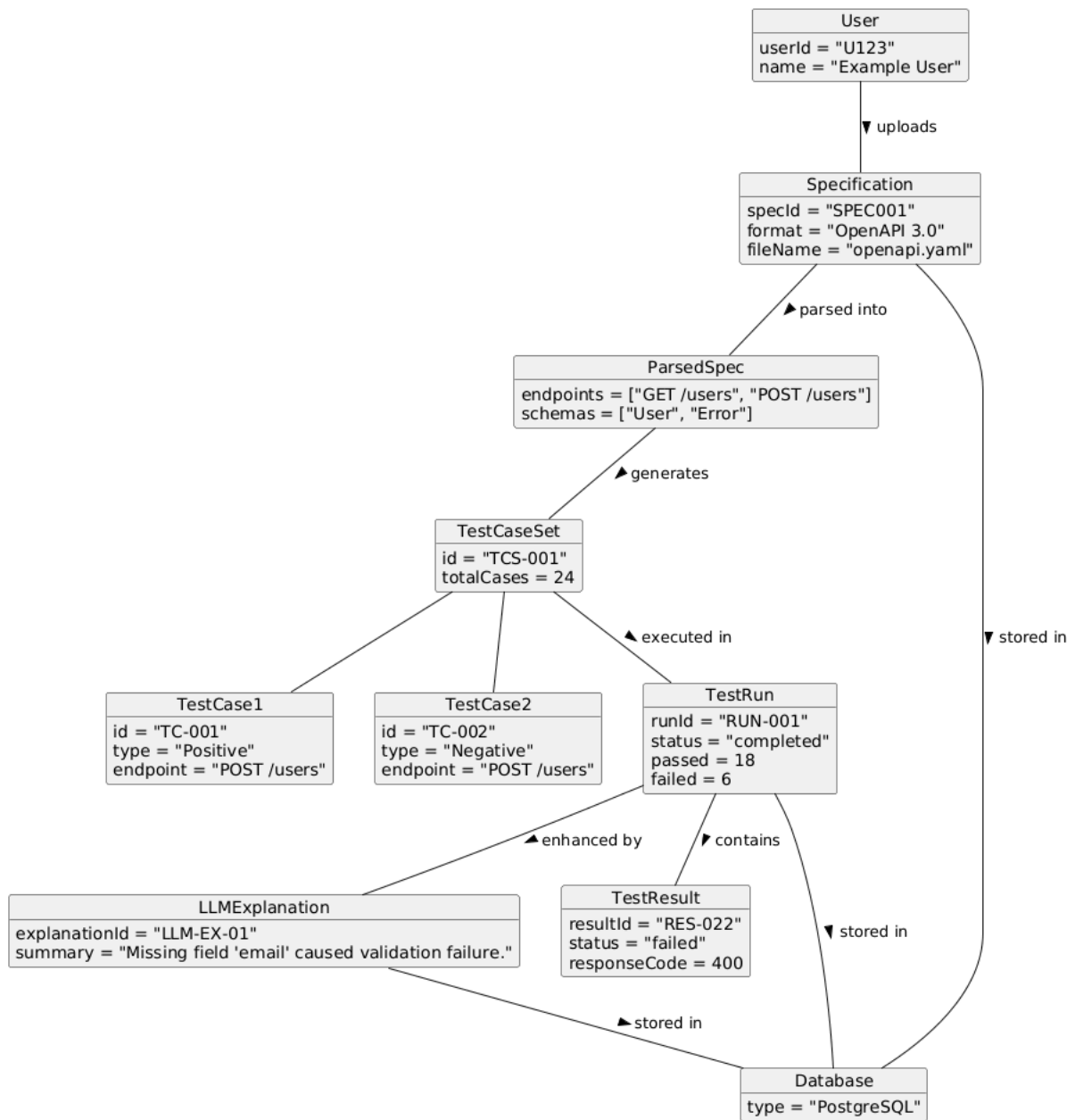
DFD Level 1C (CI/CD Workflow)



5.2 Sequence Diagram (Core Workflow)

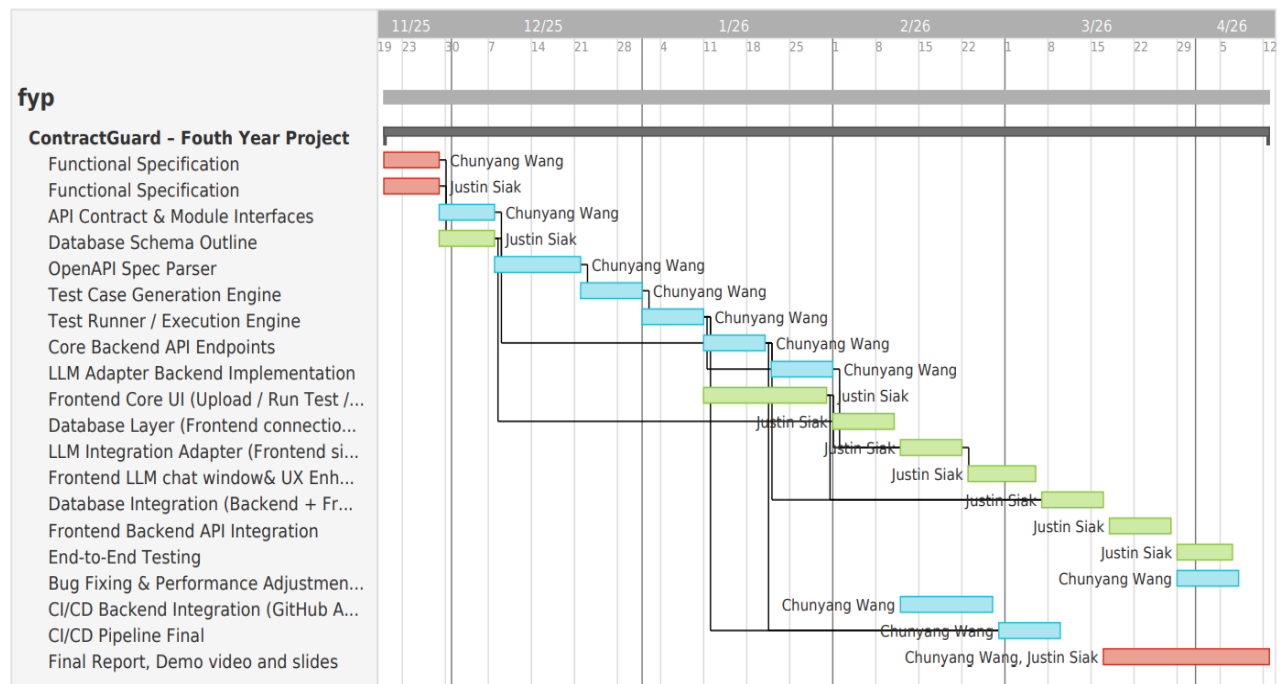


5.3 Object Model Diagram



6. Preliminary Schedule

6.1 Preliminary Schedule GANTT chart



Short Resource Requirements Description

Hardware:

The project only requires standard developer laptops capable of running Docker containers and local development tools.

Software:

Key tools include Python (FastAPI, PyTest), PostgreSQL, React, GitHub Actions, and Docker/Docker Compose, along with access to a third-party LLM API.

Wetware:

Two team members share the workload:

- Chunyang focusing on backend and CI/CD
- Justin focusing on frontend and LLM integration
- with both contributing to testing and final reporting.

7. Appendices

This section lists external references that support the concepts, standards, and technologies used in ContractGuard.

- [1]** OpenAPI Initiative, “OpenAPI Specification v3.1.0,” 2021. [Online]. Available: <https://spec.openapis.org/oas/latest.html>
- [2]** JSON Schema Organization, “Understanding JSON Schema,” 2020. [Online]. Available: <https://json-schema.org/understanding-json-schema>
- [3]** GitHub, “GitHub Actions Documentation,” 2024. [Online]. Available: <https://docs.github.com/actions>
- [4]** K. Reitz et al., “Requests: HTTP for Humans,” 2023. [Online]. Available: <https://requests.readthedocs.io>
- [5]** PyTest Developers, “pytest Documentation,” 2023. [Online]. Available: <https://docs.pytest.org>
- [6]** OpenAI, “OpenAI API Documentation,” 2024. [Online]. Available: <https://platform.openai.com/docs>
- [7]** Anthropic, “Claude API Documentation,” 2024. [Online]. Available: <https://docs.anthropic.com>
- [8]** PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2024. [Online]. Available: <https://www.postgresql.org/docs/>